and other quirks. The Unicode special mappings are conditional; they only apply under certain circumstances, such as system locale settings; otherwise, the standard specifies falling back to normal rules. When it comes to the locale-based problem of converting certain Turkish characters, `mb_convert_case()` (by design) ignores locale settings and only applies the normal mappings:

```
/*
 * https://www.unicode.org/Public/15.1.0/ucd/SpecialCasing.txt
 * Format:  <code>; <lower>; <title>; <upper>; (<condition_list>;)? #
   <comment>
 * I > ı    0049; 0131; 0049; 0049; tr Not_Before_Dot; # LATIN CAPITAL
LETTER I
 * i > İ    0069; 0069; 0130; 0130; tr; # LATIN SMALL LETTER I
 *
 * https://www.unicode.org/Public/15.1.0/ucd/UnicodeData.txt
 * Format:  <code>;<name>;2;3;4;5;6;7;8;9;10;11;<upper>;<lower>;<title>
 * I > i    0049;LATIN CAPITAL LETTER I;Lu;0;L;;;;;N;;;;0069;
 * i > I    0069;LATIN SMALL LETTER I;Ll;0;L;;;;;N;;;0049;;0049
 */

setlocale(LC_ALL, 'tr_TR.UTF-8'); // assumes system locale is installed

print "Test that i maps to \u{130}: "
    . mb_convert_case('i', MB_CASE_UPPER) . PHP_EOL;
print "Test that I maps to \u{131}: "
    . mb_convert_case("I", MB_CASE_LOWER);

Output:
Test that i maps to İ: I
Test that I maps to ı: i
```

The code above contains a comment block detailing the Unicode standard's normal and special rules as defined in some important files: the `UnicodeData.txt` and `SpecialCasing.txt` machine-readable mapping files are made publicly available via https://www.unicode.org and are intended for use in programming languages, system libraries, and so on. PHP's `mbstring` source code is compiled against these mapping files, and this is what controls `mb_convert_case()` behavior. We decode those rules as follows: The special rule when lowercasing `I` to `ı` is expressed as `0049; 0131; 0049;`